SNA Fall 2025: Assignment 4

Shuting Yang

03 December 2025

Contents

Assignment 4 Description

Libraries and Data

Libraries

```
library(igraph)
library(igraphdata)
library(intergraph)
library(statnet)
library(kableExtra)
library(ggplot2)
```

Data

```
load("luke.datasets.RData")
```

Explore

```
df <- data.frame(middleschool)
head(df, 10)
```

```
##    .tail .head
## 1      1     7
## 2      1     8
## 3      1    32
## 4      2     7
## 5      2    21
## 6      2    30
## 7      2    32
## 8      2    37
## 9      3     2
## 10     3    17
```

Task 1: Network Description

Description

```
describe.middleschool()
```

Details

```
class(middleschool)
```

```
## [1] "network"
```

```
network.size(middleschool)
```

```
## [1] 37
```

```
network.edgecount(middleschool)
```

```
## [1] 179
```

```
is.directed(middleschool)
```

```
## [1] TRUE
```

```
list.vertex.attributes(middleschool)
```

```
## [1] "gender"       "na"           "vertex.names"
```

```
list.edge.attributes(middleschool)
```

```
## [1] "na"
```

```
gender_attribute <- get.vertex.attribute(middleschool, "gender")
table(gender_attribute) #(1=female, 2=male)
```

```
## gender_attribute
##  1  2
## 22 15
```

Visualization

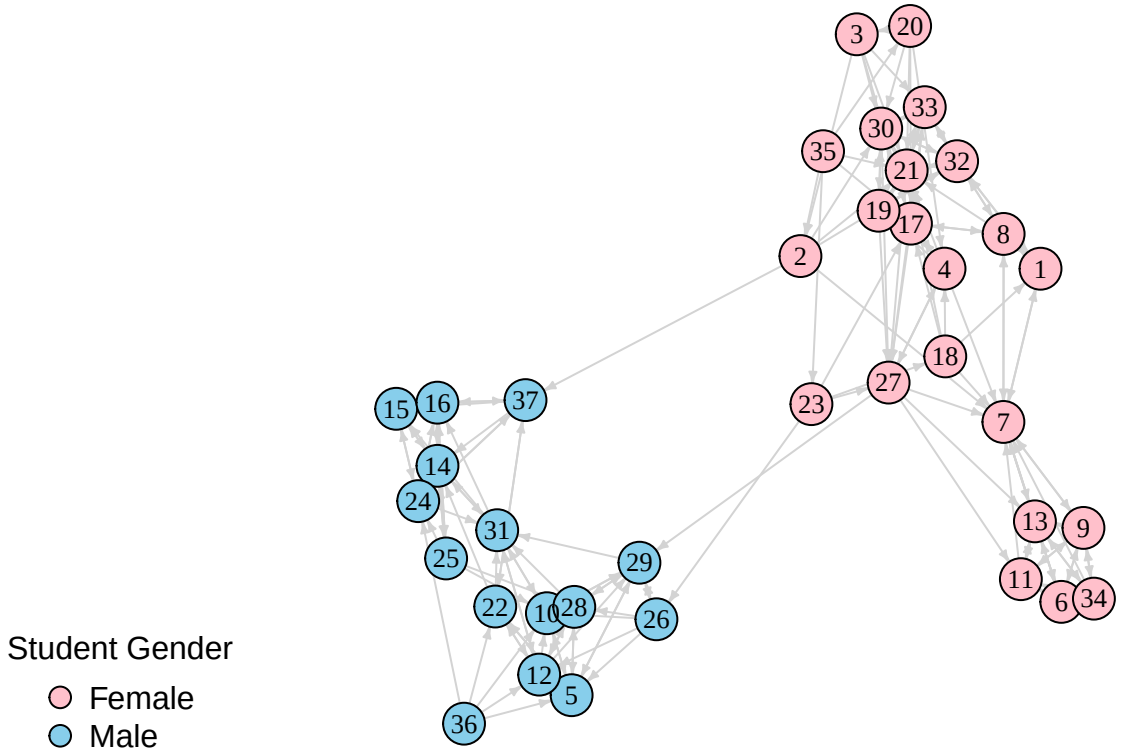
```r
set.seed(42)

g_middleschool <- graph_from_data_frame(d = middleschool, directed = TRUE)
V(g_middleschool)$gender <- middleschool %v% "gender"
V(g_middleschool)$color <- ifelse(V(g_middleschool)$gender == "2", "skyblue", "pink")

plot(g_middleschool,
     vertex.size = 12,
     vertex.label.color = "black",
     vertex.label.cex = 0.8,
     vertex.color = V(g_middleschool)$color,
     edge.arrow.size = 0.3,
     edge.arrow.width = 1,
     edge.color = "light gray",
     main = "Fig 1. Middle School Friendship Network")

legend("bottomleft",
       legend = c("Female", "Male"),
       pch = 21,
       pt.cex = 1.5,
       pt.bg = c("pink", "skyblue"),
       title = "Student Gender",
       bty = "n")
```

Fig 1. Middle School Friendship Network



Task 2: Descriptive Measures

Compute

```
# in degree
in_degrees <- igraph::degree(g_middleschool, mode="in")
most_popular_student <- V(g_middleschool)$name[which(in_degrees == max(in_degrees))]
most_popular_student
```

```
## [1] "7"  "21"
```

```
# betweenness centrality: female
node_betweenness <- igraph::betweenness(g_middleschool, normalized = TRUE)

bridges_female <- node_betweenness[V(g_middleschool)$gender == 1]
bridges_female <- names(bridges_female)[which.max(bridges_female)]
bridges_female
```

```
## [1] "27"
```

```
# betweenness centrality: male
bridges_male <- node_betweenness[V(g_middleschool)$gender == 2]
bridges_male <- names(bridges_male)[which.max(bridges_male)]
bridges_male
```

```
## [1] "29"
```

```
# closeness centrality: female
node_closeness <- igraph::closeness(g_middleschool, normalized = TRUE)

female_scores <- node_closeness[V(g_middleschool)$gender == 1]
central_members_female <- names(female_scores)[which.max(female_scores)]
central_members_female
```

```
## [1] "27"
```

```
# closeness centrality: male
male_scores <- node_closeness[V(g_middleschool)$gender == 2]
central_members_male <- names(male_scores)[which.max(male_scores)]
central_members_male
```

```
## [1] "22"
```

```
# density
edge_density(g_middleschool)
```

```
## [1] 0.1343844
```

```
# clustering coefficient
transitivity(g_middleschool, type = "average")
```

```
## [1] 0.5648688
```

```
# newman assortativity
assortativity_nominal(g_middleschool,
                      types = as.integer(V(g_middleschool)$gender),
                      directed = TRUE)
```

```
## [1] 0.9653839
```

Task 2: Subgroups - Group 1 (k-cores)

Define

A k-core is a maximal subgraph where each vertex is connected to at least k other vertices in the subgraph (Luke, 2015, p.110). K specifies the lowest number of connections (neighbors) a vertex must have to belong to the core.

Because of the nested structure of k-cores, it works by progressively removing vertices with a degree less than k in turn, and this process continues until no such vertices remain (Luke, 2015, p.112). A vertex in a k-core is also part of all lower-level cores, but the reverse is not true. This process partitions the network into dense subgroups (Nooy, Mrvar, & Batagelj, 2011).

k-cores have a number of advantages: they are nested (every member of a 4-core is also a member of a 3-core, and so on), they do not overlap, and they are easy to identify (Luke, 2015, p.110). However, the k-cores method only identifies a "dense cluster of nodes" (core vs. periphery), and it doesn't effectively split the network into different subgroups. In other words, it identifies who is "most connected," regardless of which group they belong to. For example, in the Karate network, individuals from two different factions are structurally tightly connected (refer to tutorial cohesion_part1). Secondly, unlike the more complex Community Detection algorithms, k-cores rely entirely on the pattern of internal ties and do not formally consider the pattern of ties between different groups (Luke, 2015, p.115). Therefore, they may be less effective when a subgroup is defined not only by its internal density but also by its separation from other parts of the network.

This method is especially suitable for very dense or very large networks (Nooy, Mrvar, & Batagelj, 2011, p.84). The analysis usually proceeds by progressively "peeling away" lower-level k-cores to allow researchers to focus on studying and interpreting the internal patterns of the remaining, most tightly connected core structures (Luke, 2015, p.112).

Compute

```
cores <- coreness(g_middleschool)
cores
```

```
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
##  6  5  5  7  8  8  8  7  8  8  8  8  8  8  8  8  7  5  7  5  7  8  5  8  7  6
## 27 28 29 30 31 32 33 34 35 36 37
##  7  8  8  7  8  7  7  8  5  5  8
```

```
table(cores)
```

```
## cores
##  5  6  7  8
##  7  2 10 18
```

Visualize

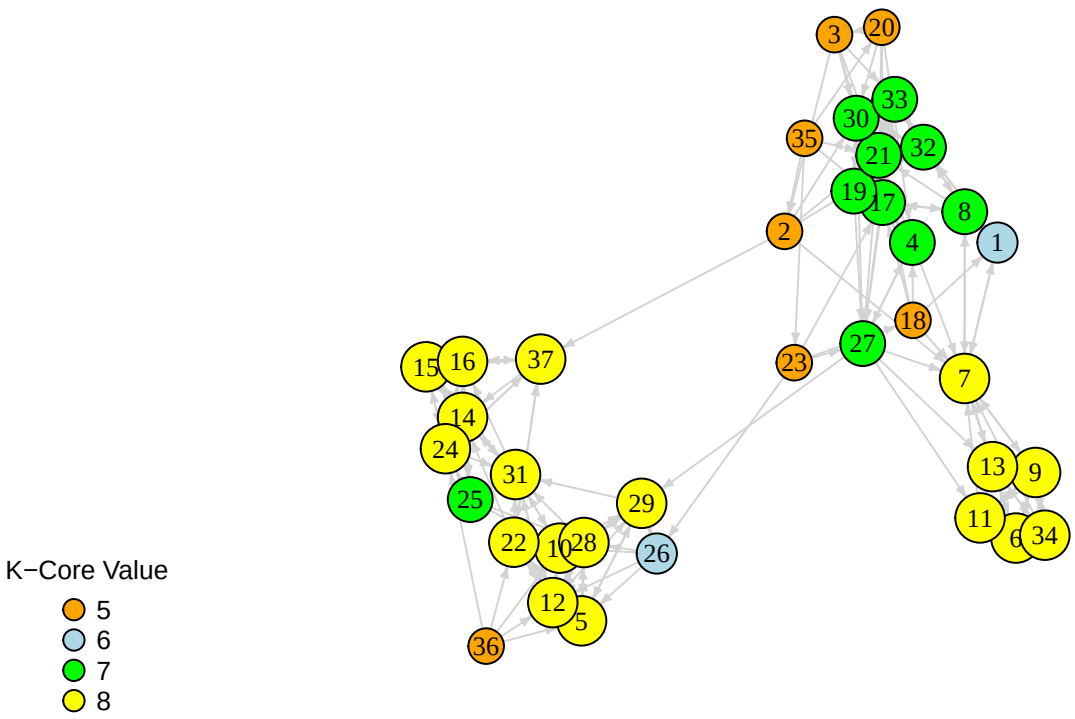
```
min_core <- min(cores)
max_core <- max(cores)
num_colors <- max_core - min_core + 1
core_palette <- colorRampPalette(c("orange", "lightblue", "green", "yellow"))(num_colors)

set.seed(42)
plot(g_middleschool,
     vertex.label.cex = 0.8,
     vertex.label.color = "black",
```

```
     vertex.size = cores * 1.5 + 4,
     vertex.color = core_palette[cores - min_core + 1],
     edge.arrow.size = 0.3,
     edge.arrow.width = 1,
     edge.color = "light gray",
     main = "Fig 2. Middle School Friendship Network (K-Core Analysis)",
     margin = c(0.1, 0.1, 0.1, 0.1))

legend("bottomleft",
       legend = min_core:max_core,
       pch = 21,
       pt.cex = 1.5,
       pt.bg  = core_palette,
       title = "K-Core Value",
       bty = "n",
       cex = 0.8)
```

Fig 2. Middle School Friendship Network (K−Core Analysis)




```
set.seed(42)
plot_kcores <- function(graph, cores) {

  # induced subgraphs
  cores_all <- induced_subgraph(graph, vids = which(cores >= 5))
  cores_6 <- induced_subgraph(graph, vids = which(cores >= 6))
  cores_7   <- induced_subgraph(graph, vids = which(cores >= 7))
  cores_8   <- induced_subgraph(graph, vids = which(cores >= 8))
```

```
  # set vertex colors as attributes
  V(cores_all)$color <- "orange"
  V(cores_6)$color <- "lightblue"
  V(cores_7)$color   <- "green"
  V(cores_8)$color   <- "yellow"

  op <- par(mfrow = c(2, 2), mar = c(3,3,3,3), cex.main = 1.5)


  # separate layouts
  plot(cores_all,
       layout = layout_with_fr(cores_all),
       main   = "Fig 3.1 k-cores all",
       vertex.label = NA,
       vertex.size = 10,
       edge.arrow.size = 0.25,
       edge.arrow.width = 1,
       edge.color = "light gray")

  plot(cores_6,
       layout = layout_with_fr(cores_6),
       main   = "Fig 3.2 k-cores 6",
       vertex.label = NA,
       vertex.size = 10,
       edge.arrow.size = 0.25,
       edge.arrow.width = 1,
       edge.color = "light gray")

  plot(cores_7,
       layout = layout_with_fr(cores_7),
       main   = "Fig 3.3 k-cores 7",
       vertex.label = NA,
       vertex.size = 10,
       edge.arrow.size = 0.25,
       edge.arrow.width = 1,
       edge.color = "light gray")

  plot(cores_8,
       layout = layout_with_fr(cores_8),
       main   = "Fig 3.4 k-cores 8",
       vertex.label = NA,
       vertex.size = 10,
       edge.arrow.size = 0.25,
       edge.arrow.width = 1,
       edge.color = "light gray")

  par(op)
}

cores <- coreness(g_middleschool)
plot_kcores(g_middleschool, cores)
```

Fig 3.1 k−cores all

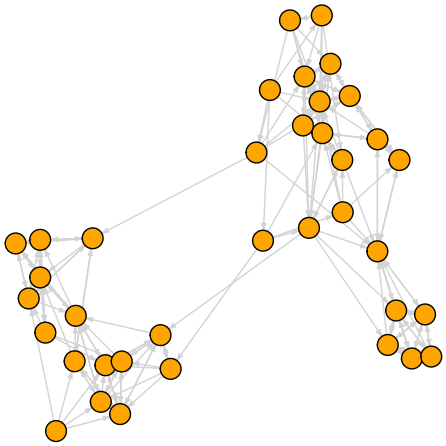


Fig 3.2 k−cores 6

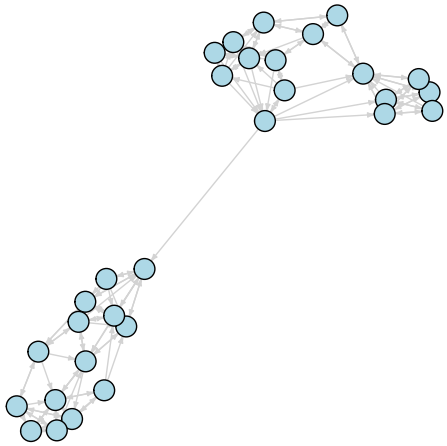


Fig 3.3 k−cores 7

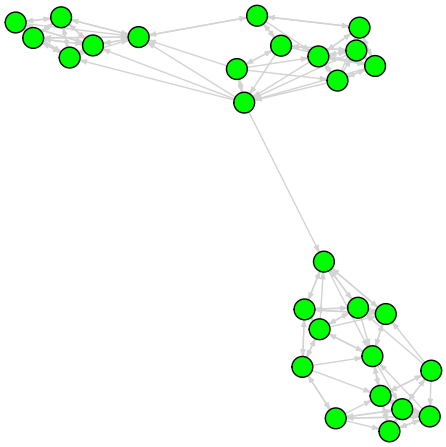
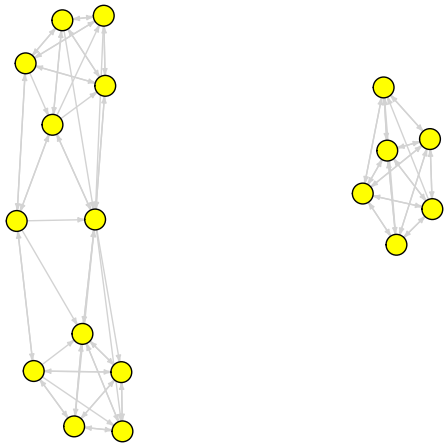


Fig 3.4 k−cores 8



Task 2: Subgroups - Group 2 (modularity)

Define

Modularity is a measure of the structure of a network that quantifies the extent to which nodes exhibit clustering. Specifically, it identifies partitions where there is greater tie density within the clusters and less density between them (Luke, 2015, p. 115). It serves as a standard metric (Lecture 7 Slides, p.38) to define how well a network can be divided into cohesive subgroups.

We mainly focus on two specific algorithms that optimize modularity: louvain and edge betweenness (Lecture 7 Slides, p.37). Louvain operates in two phases: first, it places every node in a community and moves them to neighboring communities if it results in a positive modularity gain (local optimization); second, it aggregates the identified communities into single nodes (community aggregation) and repeats the process on the new network (Lecture 7 Slides, p.47). Edge Betweenness is a divisive method that detects communities by progressively removing edges from the network (Lecture 7 Slides, p.52). It defines "betweenness" for edges as the number of shortest paths between pairs of vertices that run along it (Newman & Girvan, 2004, p.3); edges with high betweenness scores are identified as those linking different communities and are iteratively removed to reveal the underlying community structure.

Modularity method works by defining a symmetric matrix of edge fractions and calculating a score (Q) that subtracts the expected random connections from the actual within-group connections ($Q = \sum(e_{ii} - a_i^2)$). The resulting statistic ranges from -0.5 to +1; values closer to 1 indicate that the network exhibits strong clustering with respect to the grouping, while a value of 0 indicates no better than random grouping (Luke, 2015, p.115).

A major strength is that it provides an objective metric for choosing the optimal number of communities into which a network should be divided (Newman & Girvan, 2004, p.1). However, there are two main problems: one is that we need to generate a "reliable" random network for comparison, as the degree distribution of a standard random graph is often not a good model for many real-world datasets (Lecture 7 Slides, p. 42); another is resolution limit, meaning modularity is not very effective at finding communities that are very small compared to the overall size of the network (Lecture 7 Slides, p. 44).

Modularity is most appropriately used in an exploratory fashion, where algorithms (like Louvain) maximize this score to discover the best unknown node classification, or in a descriptive fashion to measure how well a specific known attribute (like gender) explains observed clustering (Luke, 2015, p. 115). It is ideal for identifying internally cohesive subgroups that are separated from other parts of the network.

Compute & Visualize

```
g_undirected <- as.undirected(g_middleschool, mode="collapse")
g_undirected
```

```
## IGRAPH 47e36d2 UN-- 37 125 --
## + attr: name (v/c), gender (v/n), color (v/c)
## + edges from 47e36d2 (vertex names):
##  [1] 2 --3  1 --7  2 --7  4 --7  6 --7  1 --8  7 --8  6 --9  7 --9  5 --10
## [11] 6 --11 7 --11 9 --11 5 --12 10--12 6 --13 7 --13 9 --13 11--13 14--15
## [21] 14--16 15--16 3 --17 4 --17 8 --17 1 --18 4 --18 7 --18 17--18 4 --19
## [31] 3 --20 4 --20 17--20 2 --21 3 --21 4 --21 8 --21 18--21 19--21 20--21
## [41] 5 --22 12--22 14--22 17--23 18--23 14--24 15--24 16--24 10--25 14--25
## [51] 16--25 24--25 5 --26 10--26 12--26 23--26 4 --27 7 --27 11--27 13--27
## [61] 17--27 19--27 21--27 23--27 5 --28 10--28 12--28 25--28 26--28 5 --29
## [71] 10--29 12--29 26--29 27--29 28--29 2 --30 3 --30 17--30 19--30 20--30
## + ... omitted several edges
```

Example 1: Louvain

```
set.seed(1)
louv <- cluster_louvain(g_undirected)
class(louv)
```

```
## [1] "communities"
```

```
modularity(louv)
```

```
## [1] 0.527776
```

```
length(louv)
```

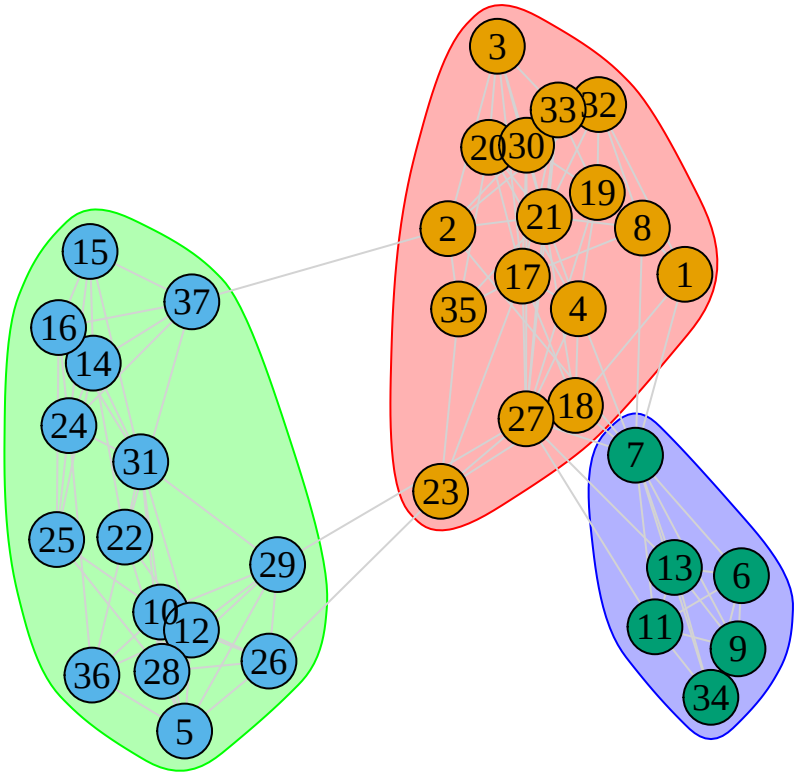
```
## [1] 3
```

```
sizes(louv)
```

```
## Community sizes
##  1  2  3
## 16 15  6
```

```
set.seed(42)
plot(louv,
     g_undirected,
     vertex.label.cex = 1.2,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main = "Fig 4. Middle School Network - Louvain Communities")
```

Fig 4. Middle School Network − Louvain Communities




```
membership(louv)
```

```
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
##  1  1  1  1  2  3  3  1  3  2  3  2  3  2  2  2  1  1  1  1  1  2  1  2  2  2
## 27 28 29 30 31 32 33 34 35 36 37
##  1  2  2  1  2  1  1  3  1  2  2
```

```
communities(louv)
```

```
## $`1`
##  [1] "1"  "2"  "3"  "4"  "8"  "17" "18" "19" "20" "21" "23" "27" "30" "32" "33"
## [16] "35"
##
## $`2`
##  [1] "5"  "10" "12" "14" "15" "16" "22" "24" "25" "26" "28" "29" "31" "36" "37"
##
## $`3`
## [1] "6"  "7"  "9"  "11" "13" "34"
```

Change resolution parameter:

```
op <- par(mfrow = c(2, 2), mar = c(3,3,3,3), cex.main = 1.5)

# 0.1
set.seed(1)
small_gamma <- cluster_louvain(g_undirected, resolution = 0.1)
set.seed(42)
plot(small_gamma,
     g_undirected,
     vertex.label.cex = 1.2,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main = "Fig 5.1 Louvain Communities (gamma = 0.1)")

# 0.5
set.seed(1)
medium_gamma <- cluster_louvain(g_undirected, resolution = 0.5)
set.seed(42)
plot(medium_gamma,
     g_undirected,
     vertex.label.cex = 1.2,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main = "Fig 5.2 Louvain Communities (gamma = 0.5)")

# 1
set.seed(1)
default_gamma <- cluster_louvain(g_undirected)
set.seed(42)
plot(default_gamma,
     g_undirected,
     vertex.label.cex = 1.2,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main = "Fig 5.3 Louvain Communities (gamma = 1)")

# 1.5
set.seed(1)
big_gamma <- cluster_louvain(g_undirected, resolution = 1.5)
set.seed(42)
plot(big_gamma,
     g_undirected,
     vertex.label.cex = 1.2,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main = "Fig 5.4 Louvain Communities (gamma = 1.5)")
```

Fig 5.1 Louvain Communities (gamma = 0.1)

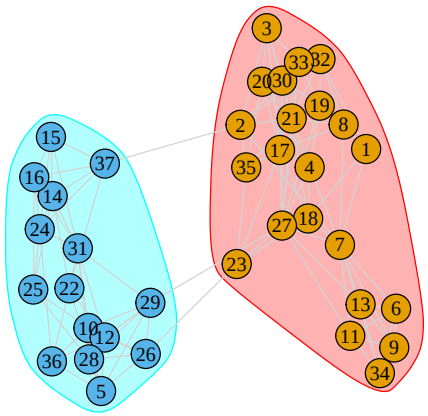


Fig 5.2 Louvain Communities (gamma = 0.5)

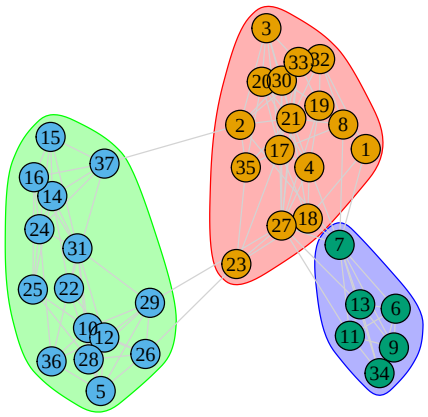


Fig 5.3 Louvain Communities (gamma = 1)

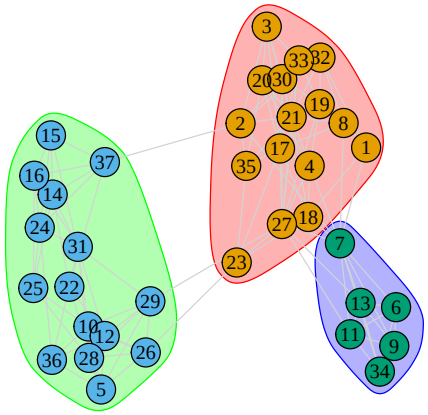
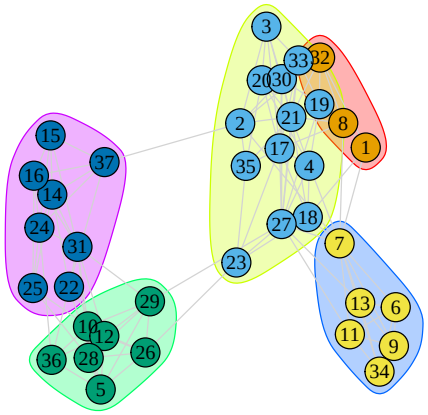


Fig 5.4 Louvain Communities (gamma = 1.5)



```
par(op)
```

```
data.frame(
  "gamma_value" = c("small_gamma", "medium_gamma", "default_gamma", "big_gamma"),
  "modularity" = c(modularity(small_gamma),
                   modularity(medium_gamma),
                   modularity(default_gamma),
                   modularity(big_gamma)),
  "number_of_communities" = c(length(small_gamma),
                              length(medium_gamma),
                              length(default_gamma),
                              length(big_gamma)),
  "community_sizes" = c(toString(sizes(small_gamma)),
                        toString(sizes(medium_gamma)),
                        toString(sizes(default_gamma)),
                        toString(sizes(big_gamma)))
)
```

```
##     gamma_value modularity number_of_communities community_sizes
## 1   small_gamma  0.9238368                      2          22, 15
## 2  medium_gamma  0.7198880                      3       16, 15, 6
## 3 default_gamma  0.5277760                      3       16, 15, 6
```

```
## 4     big_gamma  0.3836640                      5  3, 13, 7, 6, 8
```

```
table(V(g_undirected)$gender,membership(small_gamma))
```

```
##
##      1  2
##   1 22  0
##   2  0 15
```

```
table(V(g_undirected)$gender,membership(medium_gamma))
```

```
##
##      1  2  3
##   1 16  0  6
##   2  0 15  0
```

```
table(V(g_undirected)$gender,membership(default_gamma))
```

```
##
##      1  2  3
##   1 16  0  6
##   2  0 15  0
```

```
table(V(g_undirected)$gender,membership(big_gamma))
```

```
##
##      1  2  3  4  5
##   1  3 13  0  6  0
##   2  0  0  7  0  8
```

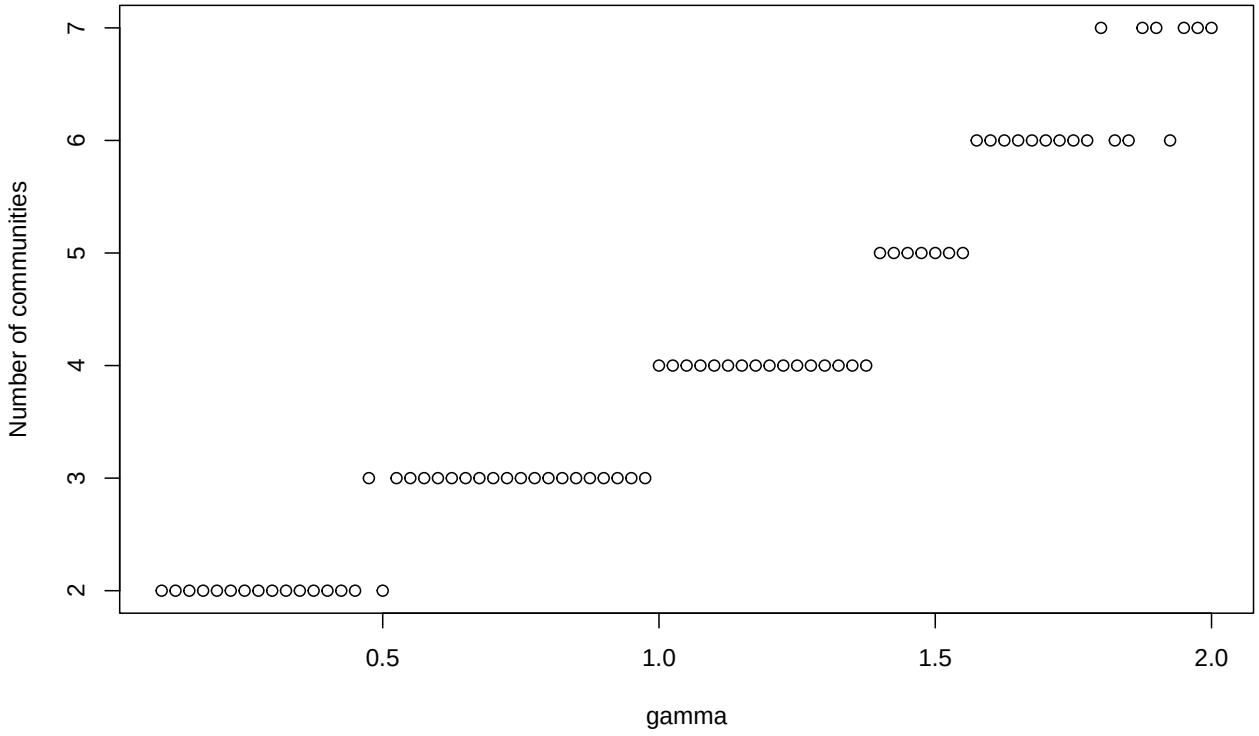
```
# create vector of gamma values of from 0.1 to 2
gamma <- seq(from = 0.1, to = 2, by = 0.025)

# empty vector to store number of communities found at each gamma level
nc <- vector("numeric",length(gamma))

# iterate over all gamma values, compute louvain each time, store results
for (i in 1:length(gamma)){
  gc <- cluster_louvain(g_undirected, resolution = gamma[i])
  nc[i] <- length(gc)
}

# visualize
plot(gamma,
     nc,
     xlab="gamma",
     ylab="Number of communities",
     main="Fig 6. Middle School Network Structure\n(Number of Communities at varying gamma values)")
```

Fig 6. Middle School Network Structure
(Number of Communities at varying gamma values)



Example 2: Edge Betweeness

```
edge <- cluster_edge_betweenness(g_undirected)
```

```
length(edge)
```

```
## [1] 4
```

```
sizes(edge)
```

```
## Community sizes
##  1  2  3  4
## 16  9  6  6
```

```
modularity(edge)
```

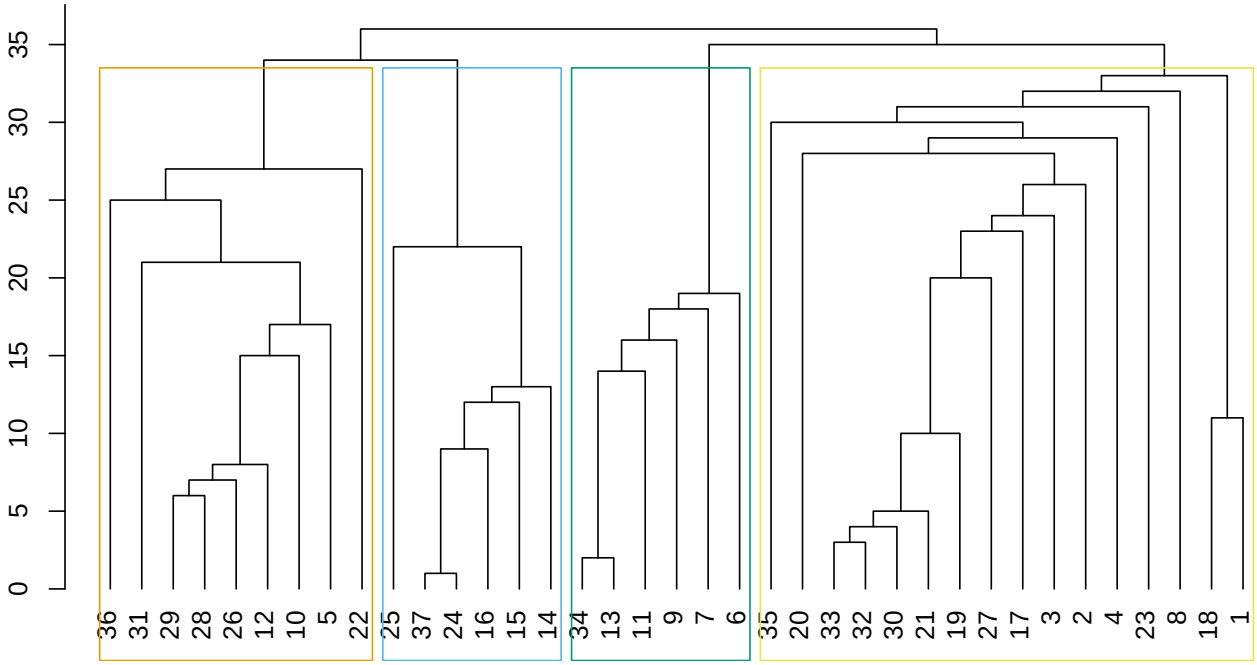
```
## [1] 0.528352
```

```
is_hierarchical(edge)
```

```
## [1] TRUE
```

```
plot_dendrogram(edge, mode = "hclust")
title(main = "Fig 7. Default Hierarchical Clustering")
```

Fig 7. Default Hierarchical Clustering




```
op <- par(mfrow = c(2, 2), mar = c(3,3,3,3), cex.main = 1.5)

plot_dendrogram(edge, mode="hclust", rect=2)
title(main = "Fig 8.1 Cut at 2")

plot_dendrogram(edge, mode="hclust", rect=3)
title(main = "Fig 8.2 Cut at 3")

plot_dendrogram(edge, mode="hclust", rect=4)
title(main = "Fig 8.3 Cut at 4")

plot_dendrogram(edge, mode="hclust", rect=5)
title(main = "Fig 8.4 Cut at 5")
```

Fig 8.1 Cut at 2

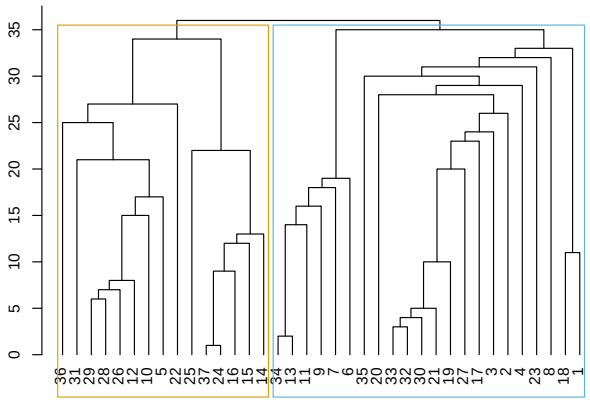


Fig 8.2 Cut at 3

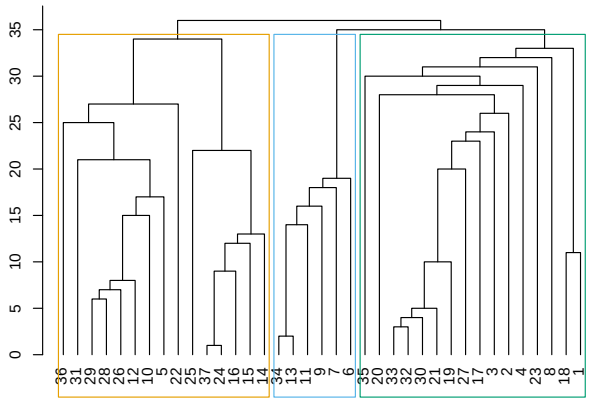


Fig 8.3 Cut at 4

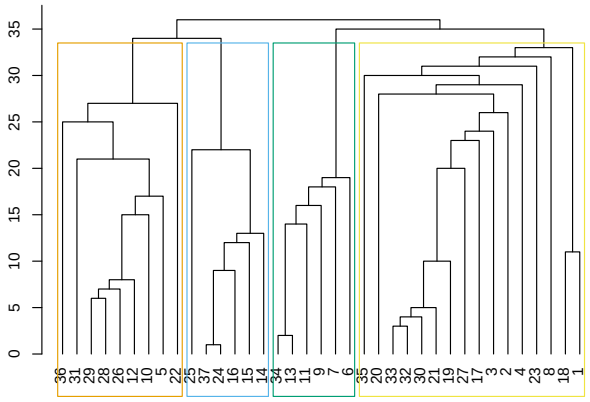
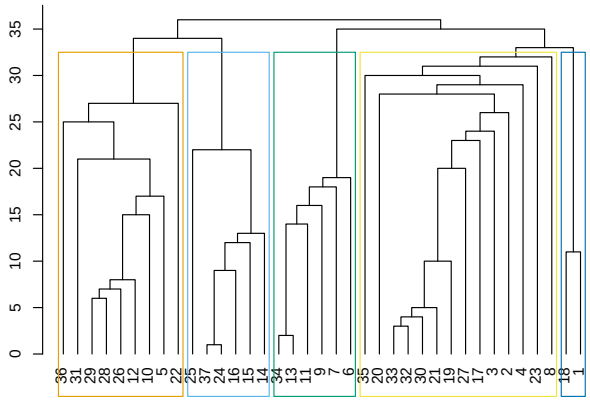


Fig 8.4 Cut at 5



```
par(op)
```

```
op <- par(mfrow = c(2, 2), mar = c(3,3,3,3), cex.main = 1.5)

g_2 <- cut_at(edge, 2)
edge$membership <- g_2
set.seed(42)
plot(edge,
     g_undirected,
     layout = layout_with_kk,
     vertex.label = V(g_undirected)$name,
     vertex.label.cex = 1,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main="Fig 9.1 Edge Betweenness (K=2)")

g_3 <- cut_at(edge, 3)
edge$membership <- g_3
set.seed(42)
plot(edge,
     g_undirected,
     layout = layout_with_kk,
     vertex.label = V(g_undirected)$name,
```

```
     vertex.label.cex = 1,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main="Fig 9.2 Edge Betweenness (K=3)")

g_4 <- cut_at(edge, 4)
edge$membership <- g_4
set.seed(42)
plot(edge,
     g_undirected,
     layout = layout_with_kk,
     vertex.label = V(g_undirected)$name,
     vertex.label.cex = 1,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main="Fig 9.3 Edge Betweenness (K=4)")

g_5 <- cut_at(edge, 5)
edge$membership <- g_5
set.seed(42)
plot(edge,
     g_undirected,
     layout = layout_with_kk,
     vertex.label = V(g_undirected)$name,
     vertex.label.cex = 1,
     vertex.label.color = "black",
     vertex.size = 16,
     edge.color = "light gray",
     main="Fig 9.4 Edge Betweenness (K=5)")
```

Fig 9.1 Edge Betweenness (K=2)

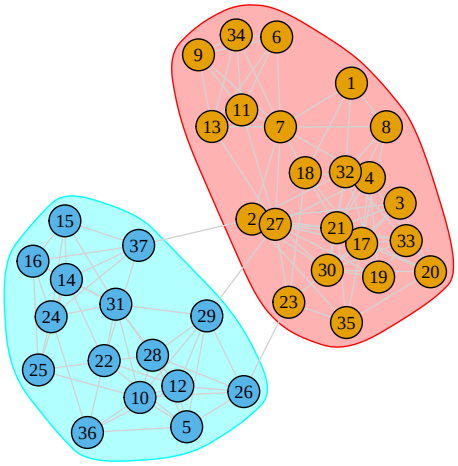


Fig 9.2 Edge Betweenness (K=3)

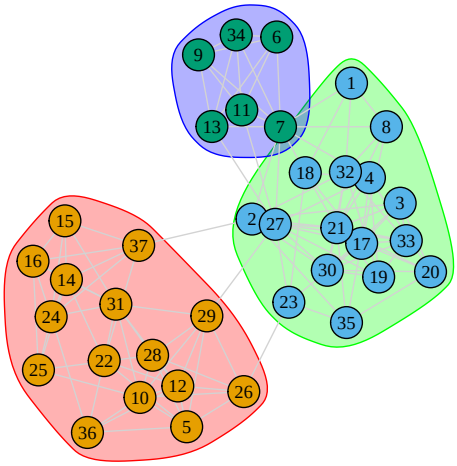


Fig 9.3 Edge Betweenness (K=4)

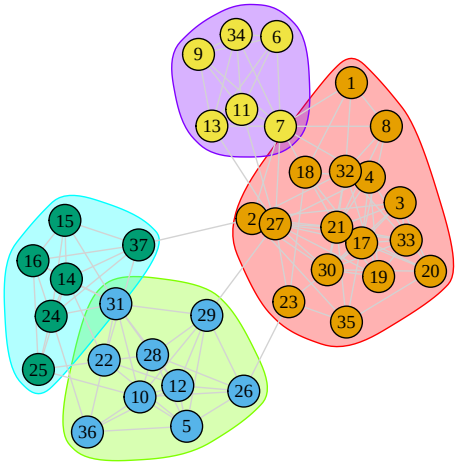
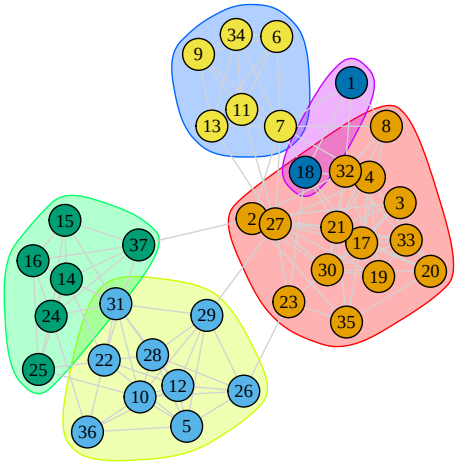


Fig 9.4 Edge Betweenness (K=5)




```
par(op)
```

```
modularity_2 <- modularity(g_undirected, g_2)
modularity_3 <- modularity(g_undirected, g_3)
modularity_4 <- modularity(g_undirected, g_4)
modularity_5 <- modularity(g_undirected, g_5)

data.frame(
  "community_cut" = c("cut_at_2", "cut_at_3", "cut_at_4", "cut_at_5"),
  "modularity" = c(modularity_2, modularity_3, modularity_4, modularity_5),
  "number_of_communities" =c(length(unique(g_2)), length(unique(g_3)), length(unique(g_4)), length(uniq
)
```

```
##   community_cut modularity number_of_communities
## 1      cut_at_2   0.454368                     2
## 2      cut_at_3   0.527776                     3
## 3      cut_at_4   0.528352                     4
## 4      cut_at_5   0.513312                     5
```

```r
extract_modularity_data <- function(communities, graph){

  # Arguments:
  # communities: igraph communities object
  # graph: igraph object

  mems_list <- list() # list where membership information will be saved
  num_communities <- NA # vector where number of communities will be saved
  modularities <- NA # avector to store modularity scores

  # Extracting levels of aggregation, or steps,
  # in the hierarchical merge data.
  num_merges <- 0:nrow(communities$merges)

  # Note that we start from 0 as the first level
  # corresponds to all nodes in their own community,
  # and that does not have a row in the  merge data frame.

  # Looping through each level of aggregation
  for (x in 1:length(num_merges)){
    # We first extract the membership
    # information at the given merge level using a
    # cut_at function. The inputs are the community
    # object and the merge step of interest.

    mems_list[[x]] <- cut_at(communities, steps = num_merges[x])

    # Now we calculate the number of communities associated
    # with the given clustering solution:
    num_communities[x] <- length(unique(mems_list[[x]]))

    # Let's also calculate the modularity score, just to make sure
    # we get the right value for that set of community memberships:
    modularities[x] <- modularity(graph, mems_list[[x]])
    }

  # We will now put together our extracted
  # information in a data frame.

  plot_data <- data.frame(modularity = modularities,
                          num_communities = num_communities)

  # Let's reorder to go from low number of communities to high:
  mems_list <- mems_list[order(plot_data$num_communities)]
  plot_data <- plot_data[order(plot_data$num_communities), ]
  rownames(plot_data) <- 1:nrow(plot_data)

  # outputting results in a list:
  return(list(summary_data = plot_data,
       membership_list = mems_list))
}
```

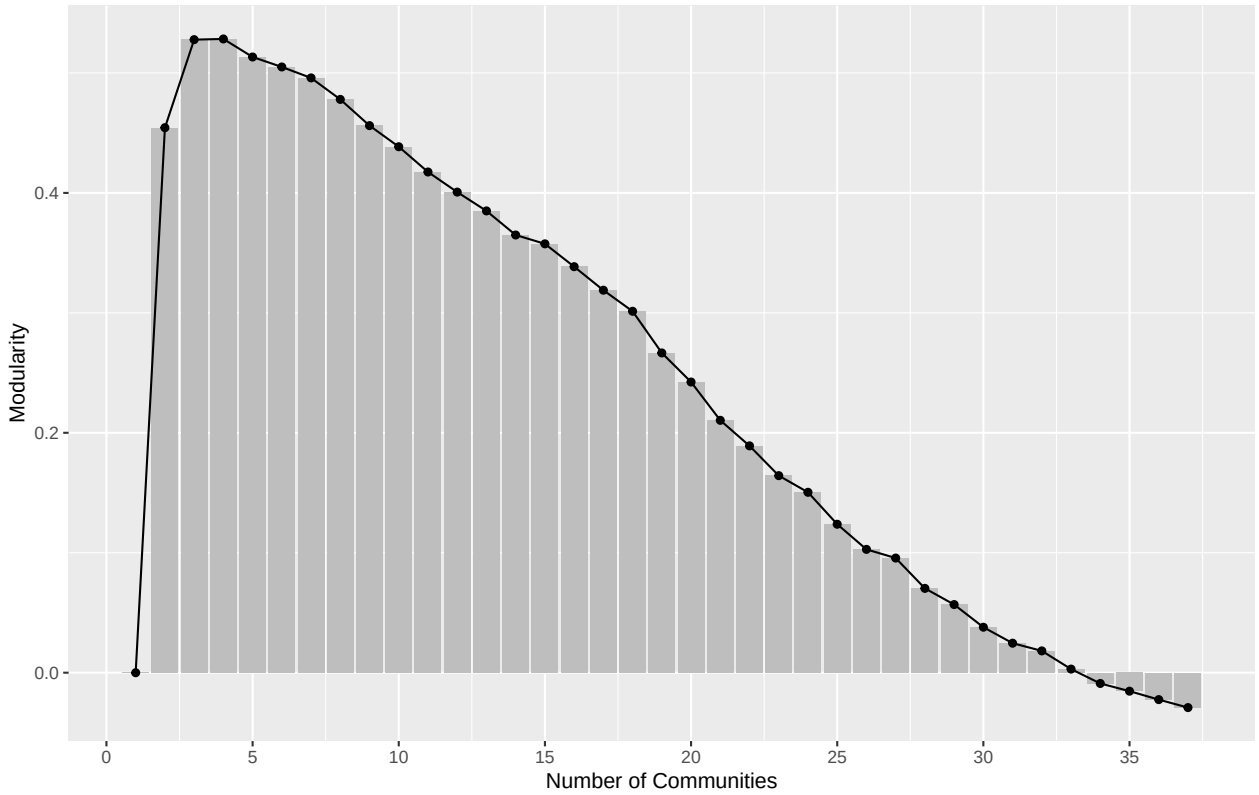
```r
modularity_data <- extract_modularity_data(communities = edge,
                                           graph = g_undirected)
```

```
summary_data <- modularity_data[[1]]
head(summary_data)
```

```
##   modularity num_communities
## 1   0.000000               1
## 2   0.454368               2
## 3   0.527776               3
## 4   0.528352               4
## 5   0.513312               5
## 6   0.504992               6
```

```
ggplot(summary_data, aes(num_communities, modularity)) +
  geom_bar(stat = "identity", fill = "grey") +
  geom_line(color = "black", linetype = "solid") +
  geom_point(shape = 19, size = 1.5, color = "black") +
  labs(title = "Fig 10. Modularity by Number of Communities",
       x = "Number of Communities", y = "Modularity") +
  scale_x_continuous(breaks = seq(0, max(summary_data$num_communities), by = 5)) +
  theme(plot.title = element_text(hjust = 0.5, size = 14))
```

Fig 10. Modularity by Number of Communities



It is clear that the two best solutions are with 3 or 4 communities, after which the modularity scores start to decrease. In general, we want solutions with high modularity, but we may not necessarily choose the one with the highest modularity, especially if adding more communities only marginally improves the fit and the simpler solution offers a more intuitive, substantively clearer story. So here, I still choose the solution with 2 communities.

```
# cut_at_2 removing 3 egdes
names(g_2) <- V(g_undirected)$name
edge_ends <- ends(g_undirected, E(g_undirected))
```

```r
crossing_indices <- which(g_2[edge_ends[,1]] != g_2[edge_ends[,2]])
bridge_nodes <- unique(as.vector(edge_ends[crossing_indices, ]))
E(g_undirected)[crossing_indices]
```

```
## + 3/125 edges from 47e36d2 (vertex names):
## [1] 23--26 27--29 2 --37
```

Task 2: Subgroups - Group 2 (blockmodels)

Define

A blockmodel is a technique that simplifies network structure by partitioning nodes into groups (blocks) based on their structural equivalence or similarity in connection patterns. Unlike community detection methods based on cohesion, blockmodels identify actors who share the same "social role" or "position" in the network, even if they are not directly connected to each other (Lecture 7 Slides, p.61). In other words, positional analysis means dividing a network into communities or subgroups based on social roles and positions rather than based on social cohesion. This approach is more grounded in sociological theory than the social cohesion approach.

The method works by four steps:

- Step 1 Partitioning. Actors are grouped into positions based on a definition of equivalence (e.g., structural equivalence), often using clustering algorithms like CONCOR (Wasserman & Faust, 1994 p.376).
- Step 2 Permutation: The rows and columns of the sociomatrix are reordered to place equivalent actors adjacent to each other, revealing rectangular "blocks" of ties (Wasserman & Faust, 1994 p.388).
- Step 3 Density Calculation: The density of ties within and between these blocks is calculated to form a density table (Wasserman & Faust, 1994 p.389).
- Step 4 Image Matrix Construction: A simplified $B \times B$ matrix (image matrix) is created to code blocks as present (1) or absent (0), which can then be visualized as a reduced graph (Wasserman & Faust, 1994 p.362).

Strengths: it reveals underlying regularities and role structures that are not visible in the raw sociomatrix, allowing for the grouping of nodes based on structural similarity rather than just cohesion (Lecture 7 Slides, p.89). Weaknesses: A major weakness is that perfect structural equivalence is rarely realized in real-world data, requiring the use of approximate measures and subjective thresholds (e.g., alpha density, lean fit) to define image matrices (Wasserman & Faust, 1994 p.366). Additionally, structural equivalence is specific to a single population, making it difficult to compare roles across different networks (e.g., across different cities) because the specific alters defining the ties are different (Wasserman & Faust, 1994 p.392).

This method is most appropriate for analyzing social roles and positions within a network, where the goal is to identify actors who have similar roles or positions to others, even if they are not directly connected (Lecture 7 Slides, p.89). It is particularly useful for simplifying complex (Wasserman & Faust, 1994 p.361), multirelational networks into interpretable structures (reduced graphs) and for detecting non-assortative structures, such as core-periphery or bipartite patterns, which standard community detection methods often miss (Shai, Stanley, Granell, Taylor, & Mucha, 2021, p.3).

Compute

```r
m <- as_adjacency_matrix(g_undirected, sparse = FALSE)
```

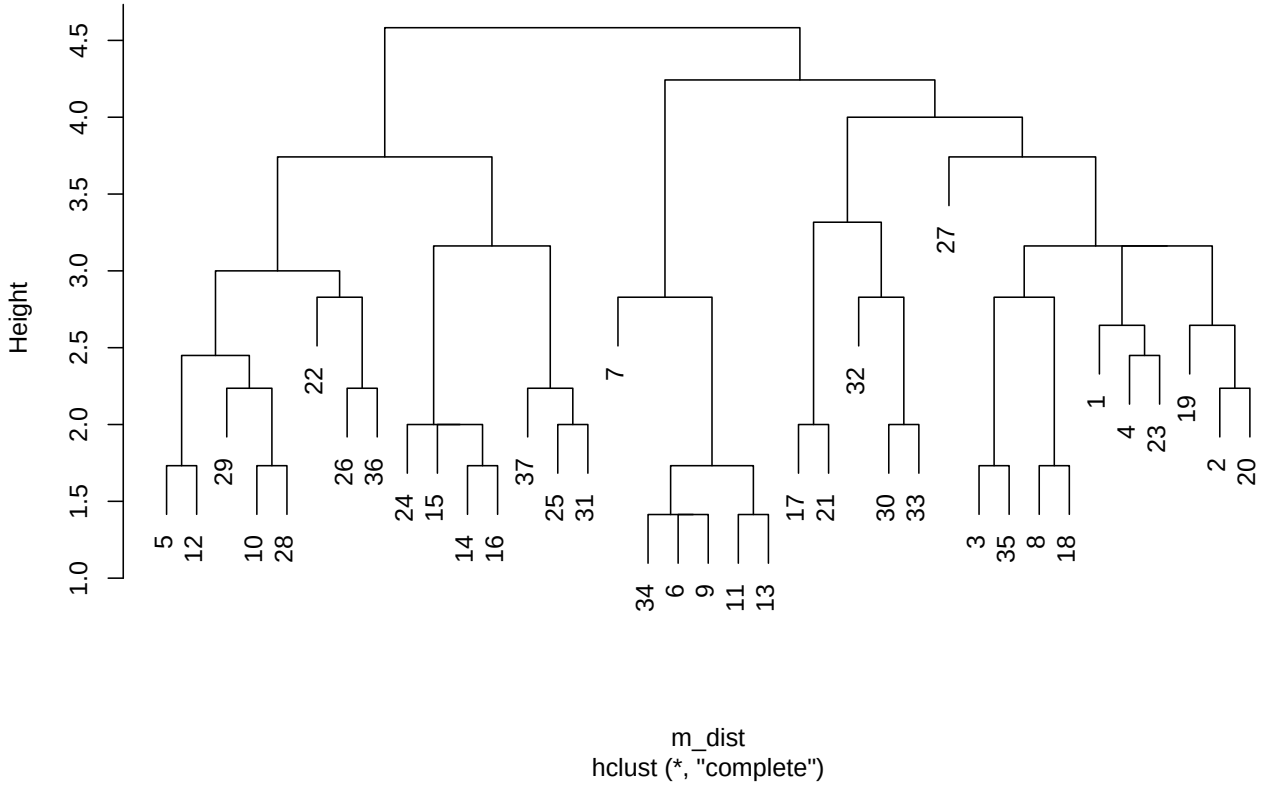
```r
# step 1
m_dist <- dist(m, method = "euclidean", upper = TRUE)
```

```r
# step 2
hc <- hclust(m_dist)
hc
```

```
##
## Call:
## hclust(d = m_dist)
##
## Cluster method   : complete
## Distance         : euclidean
## Number of objects: 37
```

```r
plot(hc, main = "Fig 11. Cluster  Dengrogram")
```

Fig 11. Cluster Dengrogram




```r
# step 3
b <- blockmodel(m, hc, k=2)
b
```

```
##
## Network Blockmodel:
##
## Block membership:
##
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
##  1  1  1  1  2  1  1  1  1  2  1  2  1  2  2  2  1  1  1  1  1  2  1  2  2  2
## 27 28 29 30 31 32 33 34 35 36 37
##  1  2  2  1  2  1  1  1  1  2  2
##
```

```
## Reduced form blockmodel:
##
##   1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
##             Block 1     Block 2
## Block 1 0.320346320 0.009090909
## Block 2 0.009090909 0.457142857
```

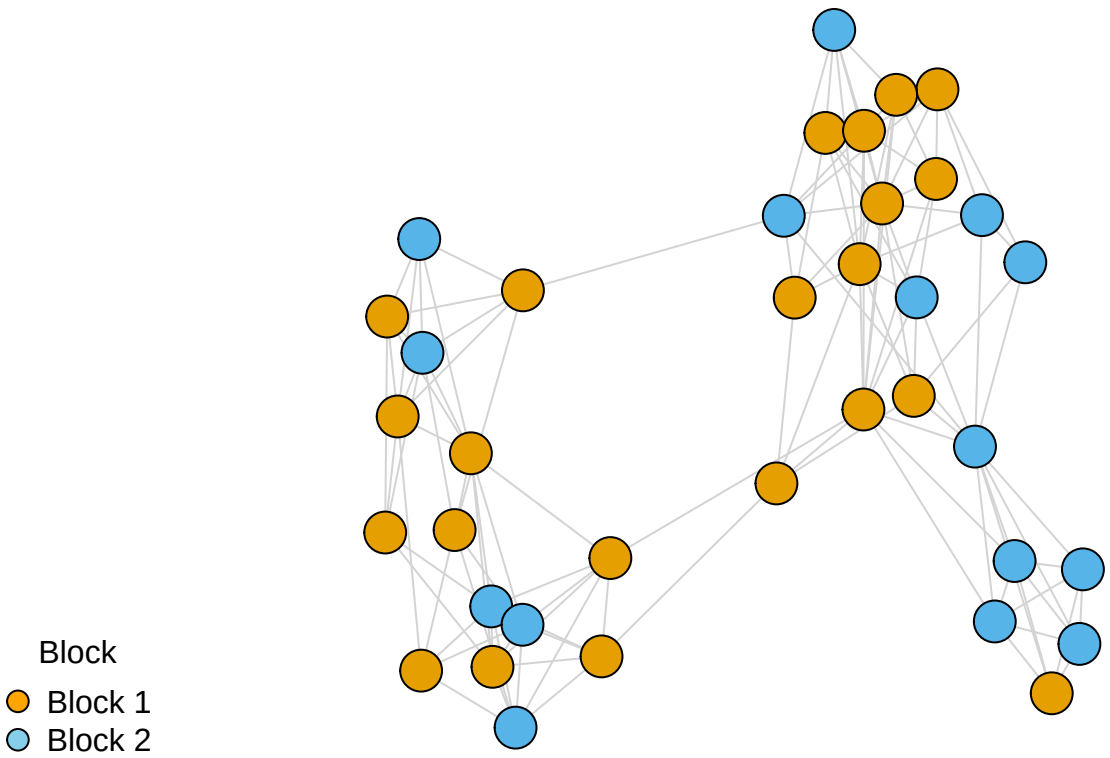
Visualize

```
V(g_undirected)$block <-  b$block.membership

set.seed(42)
plot(g_undirected,
     vertex.color = V(g_undirected)$block,
     vertex.size = 12,
     vertex.label = NA,
     edge.color = "light gray",
     main = "Fig 12. Blockmodel Visualization(k = 2)")

legend("bottomleft",
       legend = c("Block 1", "Block 2"),
       pt.bg = c("orange", "skyblue"),
       pch = 21,
       pt.cex = 1.5,
       title = "Block",
       bty = "n"
)
```

Fig 12. Blockmodel Visualization(k = 2)



Interpret

1. expectations from descriptive measures and the network paper

I expect the network to break down primarily into two distinct subgroups based on gender (boys and girls). While further subdivisions may exist within these groups (particularly among the girls), the most significant structural divide is biological sex.

First, the assortativity coefficient is 0.965, extremely close to 1, indicating that students are almost exclusively connected to others of the same gender.

Second, the overall edge density is low (0.134), suggesting the network as a whole is sparse while the transitivity (clustering coefficient) is relatively high (0.565). This discrepancy (low global density vs. high local clustering) indicates that while students are not connected to everyone in the class, they form dense local clusters ("friends of friends are friends"), which aligns with the visual separation of the pink and blue nodes in my plots.

Third, the network paper explicitly states that in this dataset, homophily is a defining feature of this specific sixth grade students: "boys overwhelmingly chose boys as friends and girls chose girls" and that there are "two groups defined by sex".

2. methods evaluation and improvement

I mainly use three methods to evaluate the subgrouping results: k-cores, modularity (specifically Louvain and Edge Betweenness), and blockmodels. Overall, the modularity methods performed the best, while k-cores performed the worst. The reasons are as follows.

First, k-cores (poor) This method failed to align with the expectation of a binary gender split. As seen in my computation, the k-core decomposition identified nested subsets of nodes (cores of 5, 6, 7, and 8) rather

than partitioning the network into distinct groups. It highlighted the most dense "centers" of the friendship circles (e.g., the core-8 group) but did not separate the network into "boys vs. girls." It revealed vertical hierarchy (core vs. periphery) rather than horizontal segmentation (group vs. group).

Second, modularity (excellent): The default algorithm (louvain) found 3 communities (modularity ~0.53). While it separated the boys into one group, it split the girls into two smaller subgroups. This is "okay" but didn't perfectly match the single binary gender split. After adjusting the resolution parameter (we can also see in the fig 6 scree plot when gamma < 0.5, it divides the nodes into two distinct groups) the method aligned perfectly with expectations. It identified exactly 2 communities (sizes 22 and 15, modularity increases to 0.92 when gamma is 0.1). The confusion matrix (table) showed a 100% overlap with gender: all 22 females were in Community 1, and all 15 males were in Community 2.

Additionally, the default hierarchical clustering (edge betweeness) found 4 communities (modularity ~0.53), which fragmented the network more than expected. However, in general, we want solutions with high modularity, we may not necessarily choose the one with the highest modularity, especially if adding more communities only marginally improves the fit and the simpler solution offers a more intuitive, substantively clearer story. So when I manually forced the dendrogram to be cut at $k = 2$ (using cut_at(edge, 2), which removes three bridge edges: 23–26 27–29 2–37), it successfully reproduced the expected gender split, isolating the male and female clusters perfectly.

Third, blockmodels (good): Using structural equivalence with a preset of $k = 2$, this method also aligned well. The visualization shows that it successfully partitioned the nodes into two blocks that correspond to the gendered groups (orange and blue clusters), suggesting that "being a boy" and "being a girl" are structurally equivalent roles in this network.

Overall, K-Cores is the worst method for this specific task, because it is designed to filter for cohesion (how connected you are) rather than division (who you are connected to). It simply peel away less connected students, leaving a mix of popular boys and girls in the high-k cores. It is a tool for identifying "coreness," not for community detection. Conversely, louvain with small gamma (e.g., gamma = 0.1) is the best method, because louvain with a lower resolution parameter naturally optimize for larger communities, revealing the macro-level gender structure. The resulting modularity (0.92) is extremely high, indicating this 2-group split is the most significant structural feature of the network.

3. a combined qualitative and quantitative approach

First based on the high assortativity coefficient (0.965) and the homophily described in the Valente paper, I qualitatively expect the network to break down into two distinct subgroups defined by gender (boys and girls).

Second, based on the results from the Louvain method's scree plot, it can be seen that when the resolution parameter is < 0.5, the network is divided into 2 groups; when $0.5 <=$ resolution parameter $<= 1$, it is divided into 3 groups; when $1 <$ resolution parameter < 1.5, it is divided into 4 groups; and when the resolution parameter $>= 1.5$, it is divided into 5 groups (note: these boundaries are approximate estimates). Therefore, I select one resolution parameter value from each interval and calculated the modularity scores for different gamma values. The calculation shows that when gamma = 0.1, the modularity is highest, reaching 0.92, quantitatively confirming that the 2-group gender split is the most dominant structural feature of the network.

Third by quantitatively analyzing the cut at $k = 2$, I identified that the algorithm removed exactly three specific edges to separate the groups: 23–26, 27–29, and 2–37. The fact that removing just these three ties (out of 179 total edges) completely disconnected the male and female clusters quantitatively demonstrates the extreme fragility of the connection between the two gender groups. At the same time, the modularity score is also very high (Figure 10) when the number of communities is two.

Overall, the exploration revealed that while default algorithms might detect smaller subgroups (e.g., splitting girls into subgroups), the most robust structural feature of the network is the binary gender divide. This is supported quantitatively by the peak modularity score in Louvain and Edge Betweenness.

a. Are the subgroups fragmented or cohesive? Which nodes stand out, and why? Are there nodes that frequently switch groups across methods? What might explain that?

The subgroups are highly cohesive internally (newman assortativity is 0.965), even though the network as a whole is fragmented into two distinct components (whole density is low, only 0.134).

First, node 27 stands out as the most critical actor in the entire network. My calculations show she holds the top rank for both Betweenness Centrality (bridge, controlling the flow of information to the boys) and Closeness Centrality (center, closest to all other girls) within the female group.

Second, node 29 stands out as the male student with the highest Betweenness Centrality, meaning that he acts as the "bridge" for the boys; without him, the connection to the female group (likely via node 27) would be significantly harder to maintain.

Third, node 22 stands out as the node with the highest Closeness Centrality among the male students. While he may not have the most direct friends (popularity), he is the most efficient at reaching all other boys in the network. Notably, this empirical finding aligns perfectly with Valente's visual analysis, which explicitly labels Node 22 as a "Central Member" in Figure 1-1.

Fourth, node 7 and node 21 are the most popular students with the highest in-degree (most friendship nominations).

There are 6 female students frequently switch groups: node 6, 7, 9, 11, 13, 34. When gamma = 0.1 or edge betweeness cut at 2, these 6 students are merged into the Group 1 (female group). However, when louvain default or edge betweeness cut at 3, this specific group splits off to form their own distinct, tight-knit subgroup. Because in the large girls group, these 6 specific girls form a much denser sub-clique internally. They are a group within a group (a nested structure).

b. Is there overlap between groups produced by different methods? Why or why not?

Yes, there is an extremely high degree of overlap between the groups produced by the Modularity (Louvain $\gamma = 0.1$) method, the Edge Betweenness ($k = 2$) method, and the Blockmodels ($k = 2$) method. From the results below, it can be seen that when k = 2, the two groups produced by Louvain, edge betweenness, and blockmodels are completely identical. However, there is little to no overlap between these partitioning methods and the K-Cores method.

The overlap exists because the network has strong gender homophily. The connection between the male and female groups relies on only three specific edges (bridging nodes 2, 23, 27), so three methods mentioned above all mathematically converge on this same "weakest link" to draw the boundary.

K-Cores do not overlap because community detection methods slice the network horizontally to find distinct groups while k-cores slice the network vertically to find depth (who is the most connected), regardless of which group they belong to.

```
# gender(female=1, male=2)
V(g_undirected)$gender
```

```
##  [1] 1 1 1 1 2 1 1 1 1 2 1 2 1 2 2 2 1 1 1 1 1 2 1 2 2 2 1 2 2 1 2 1 1 1 1 2 2
```

```
# louvain(gamma = 0.1)
small_gamma$membership
```

```
##  [1] 1 1 1 1 2 1 1 1 1 2 1 2 1 2 2 2 1 1 1 1 1 2 1 2 2 2 1 2 2 1 2 1 1 1 1 2 2
```

```
# edge betweenness(k = 2)
mems_list <- modularity_data[[2]]
mems_ids <- which(summary_data$num_communities == 2)
mems_eb2 <- mems_list[[mems_ids]]
mems_eb2
```

```
##  [1] 1 1 1 1 2 1 1 1 1 2 1 2 1 2 2 2 1 1 1 1 1 2 1 2 2 2 1 2 2 1 2 1 1 1 1 2 2
```

```r
# blockmodels(k = 2)
bm_unsorted <- b$block.membership
bm_sorted <- bm_unsorted[V(g_undirected)$name]
unname(bm_sorted)
```

```
##  [1] 1 1 1 1 2 1 1 1 1 2 1 2 1 2 2 2 1 1 1 1 1 2 1 2 2 2 1 2 2 1 2 2 1 2 1 1 1 1 2 2
```

```r
data <- data.frame(
  gender = V(g_undirected)$gender,
  louvain = small_gamma$membership,
  edge_betweenness = mems_eb2,
  blockmodels = bm_sorted
)

data$is_consistent <- (data[, 1] == data[, 2]) &
                      (data[, 2] == data[, 3]) &
                      (data[, 3] == data[, 4])

data$is_consistent
```

```
##  [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

c. Do any methods identify subgroups that appear unrealistic or inconsistent with the context of the network? Why might that happen?

Yes, k-cores and default edge betweeness(cut at 4) are inconsistent withe the network. Because k-cores group students based on popularity (degree) rather than actual friendship ties, the most popular boys and girls don't necessarily have relationships with each other. Default edge betweeness(cut at 4) over-fragmented the network into four smaller sub-cliques, failing to capture the most fundamental structural feature: the binary divide between the male and female groups.

d. How do the subgroup structures vary if you vary parameters (e.g., resolution, etc)?

Louvain: when the resolution parameter is < 0.5, the network is divided into 2 groups; when $0.5 <=$ resolution parameter $<= 1$, it is divided into 3 groups; when $1 <$ resolution parameter < 1.5, it is divided into 4 groups; and when the resolution parameter $>= 1.5$, it is divided into 5 groups. As resolution increases, the identified subgroups become smaller and more fragmented, moving from detecting the macro-structure (gender) to detecting micro-structures (cliques).

Edge Betweeness: when k = 2, the network is divided into 2 groups (male and female group); when k = 3, the female group splits into two, while the male group remains unchanged; when k = 4 or 5, both gender groups are fragmented. Increasing k progressively peels apart the nested layers of the network. The jump from $k = 2$ to $k = 3$ reveals that the female group is structurally less unified than the male group.

e. Do the subgroups correspond to node attributes (e.g., gender, role)?

Yes, in middle school network, the subgroups correspond to node gender attribute.

f. Do descriptive measures (e.g., degree, betweenness, density, etc.) help you interpret the subgroups? How? Do any methods reveal hidden or unexpected structure that the descriptive measures did not anticipate?

In-degree identified the "popular" students (nodes 7 and 21) within the subgroups, helping to interpret the internal hierarchy of the clusters. Betweenness centrality: identified the "bridges" (node 27 and 29), explaining how the two subgroups remain faintly connected rather than completely isolated. Density and Transitivity: the density is low (0.134) while the transitivity (clustering coefficient) is relatively high (0.565). This discrepancy (low global density vs. high local clustering) indicates that while students are not connected

to everyone in the class, they form dense local clusters ("friends of friends are friends"), which aligns with the visual separation of the pink and blue nodes in my plots. Assortativity: The high assortativity coefficient (0.965) means high homophily, predicting that any valid community detection (such as louvain and edge betweenness) would likely fall along gender lines.

Yes, the Louvain (default) and Edge Betweenness (default/k=4) methods revealed a hidden nested structure within the female group that descriptive measures did not anticipate. While descriptive measures (like assortativity) suggested a simple binary split (boys vs. girls), these higher-resolution methods detected a distinct subgroup of 6 girls (nodes 6, 7, 9, 11, 13, 34).

AI Disclosure Statement and Learning Reflections

I mainly used AI to help adjust the plots, such as the title size(`cex.main`) and legend shapes(`pch = 21`). I also used it to get the code to figure out exactly which nodes and edges connect the two groups.

I do have a question: In this middleschool network, it's pretty obvious that there are two groups based on gender. But why does the Edge Betweenness calculation show the highest modularity when cutting at 4? Also, I am wondering how to plot the blockmodel matrix figure, I try many times but still not very sure about that.

References

Luke, D. A., Carothers, B. J., Dhand, A., Bell, R. A., Moreland-Russell, S., Sarli, C. C., & Evanoff, B. A. (2015). Breaking Down Silos: Mapping Growth of Cross-Disciplinary Collaboration in a Translational Science Initiative. Clinical and Translational Science, 8(2), 143-149. https://doi.org/10.1111/cts.12248

Nooy, W. D., Mrvar, A., & Batagelj, V. (2011). Exploratory social network analysis with Pajek. Cambridge university press.

Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. Physical Review E, 69(2), 1–15.

Shai, S., Stanley, N., Granell, C., Taylor, D., & Mucha, P. J. (2021). Case studies in network community detection. In R. Light & J. Moody (Eds.), The Oxford Handbook of Social Networks (pp. 311-329). Oxford University Press.

Wasserman, S., & Faust, K. (1994). Social network analysis: Methods and applications (Chapter 9). Cambridge University Press.